

0.2 Wyświetl imię i nazwisko użytkownika Alice.
curl -X 'GET' 'http://127.0.0.1:5000/user/alice' -H accept:/' jq -r "name"
0.7 Sprawdź, czy jednym z hobby użytkownika Bob są gry.
curl -X 'GET' '[http://127.0.0.1:5000/user/bob]' -H accept:/' jq -r 'any(.hobbies[]; . == "games")'
0.8 Wyświetl pierwsze hobby użytkownika Alice.
curl -X 'GET' '[http://127.0.0.1:5000/user/alice]' -H accept:/' jq -r .hobbies[0]
0.9 Sprawdź, ile hobby ma użytkownik Bob.
curl -X 'GET' '[http://127.0.0.1:5000/user/bob]' -H accept:/' jq -r 'hobbies length'
0.10 Wyświetl nazwę użytkownika i miasto jako jeden ciąg znaków, np. "Alice Smith (Warsaw)". Zmodyfikuj polecenie, aby tekst był wyświetlany jako "Alice Smith from Warsaw".
curl -X 'GET' '[http://127.0.0.1:5000/user/alice]' -H accept:/' jq -r '"(.(name)) ((.(city)))"'
curl -X 'GET' '[http://127.0.0.1:5000/user/alice]' -H accept:/' jq -r '"(.(name) from \ (.(city))"'
0.12 Wyświetl przedmioty droższe niż 20 pln.
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r '[] select(.price > 20.00)'
0.14 Posortuj przedmioty według ceny rosnąco, a następnie malejąco.
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r 'sort_by(.price)'
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r 'sort_by(.price) reverse'
0.15 Pobierz sumę cen wszystkich przedmiotów.
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r 'map(.price) add'
0.16 Wyświetl przedmioty, których nazwa zawiera "item".
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r '[] select(.name test("item"))'
0.17 Wyświetl przedmioty w przedziale cenowym 10-30 pln.
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r 'map(select(.price >= 10.00 and .price <= 30.00))'
0.23 Wyświetl przedmiot o największej cenie.
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r 'sort_by(.price) reverse .[0]'
0.24 Wyświetl wszystkie przedmioty z ceną podwyższoną o 10%.
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r '[] .price=(.price*1.1)'
0.26 Wyświetl hobby Alice jako string połączony przecinkami.
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r 'hobbies join(",")'
0.27 Pobierz pierwszy i ostatni przedmiot z listy.
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' jq -r '.[0], .[-1]'

0.28 Sprawdź, czy są przedmioty droższe niż 100 pln.

```
curl -X 'GET' '[http://127.0.0.1:5000/items]' -H accept:/' | jq -r '[] | (.price > 100)'
```

Funkcje haszujące

Przydatne curl'e

curl -X GET http://localhost:port/hash (wyświetla wynik w terminalu)

```
curl -X POST http://localhost:port/submit -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"session_id":"id_sesji", "hash_hex":"przykładowy_hash_hex"}
```

Wyświetlenie listy algorytmów

openssl list -cipher-algorithms
opcjonalnie szukajka: openssl list -cipher-algorithms | grep "blowfish"

Obliczenie skrótu MD5 / SHA-256 / SHA-512 otrzymanego słowa, kodując wynik w hex

echo -n "przykładowe_słowo" | openssl dgst -md5 -SHA-256 -SHA-512

Obliczenie skrótu Argon

W kontenerze dockerowym

```
openssl kdf -keylen 24 -kdfopt pass:przykładowe_słowo -kdfopt salt:przykładowa_sól -kdfopt iter:przykładowa_iteracja -kdfopt memcost:przykładowa_pamięć ARGON2D | tr -d ':' | tr '[:upper:]' '[:lower:]'
```

Obliczenie skrótu bcrypt

użyte narzędzie htpasswd

```
-n → wyświetlanie wyniku w terminalu
-b → użycie hasła z linii komend
-B → wymusi bcrypt
```

podman run docker.io/mazurkatarzyna/htpasswd:lastest -nb username przykładowe_słowo

Użycie hash-identfier mając początkowy ciąg znaków i hash

- podman run -it docker.io/mazurkatarzyna/hash-identifier:lastest
- wklejamy hash do sprawdzenia
- dostajemy wynik w possible hashes np. SHA-1
- sprawdzamy wynik → echo -n "przykładowe słowo" | openssl dgst -sha-1

Użycie hashid (jak hash-identfier nie zadziała)

- hashid
- od nowej linii wklejamy hash
- sprawdzenie → echo -n "przykładowe_słowo" | openssl dgst -algotrytm LUB hashcat

hashcat użycie

- tworzenie pliku z hashem (może być zwykłe txt)
- tworzenie pliku ze słowem
- hashcat -m (numerek konkretnego algorytmu) -a 0 (zwykły atak słownikowy bez masek) hash.txt word.txt
- wyświetlenie wyniku → hashcat -m (numerek konkretnego algorytmu) hash.txt (początkowy plik z hashem) --show

Szyfrowanie symetryczne

Przydatne curl'e

curl -X GET http://localhost:port/encrypt -H 'accept: application/json'

```
curl -X POST http://localhost:port/submit -H 'accept: */*' -H 'Content-Type: application/json' -d '{"session_id":"id_sesji", "encrypted_b64":"zaszyfrowane_słowo_base64"}
```

```
curl -s -o data.txt -X GET 'http://localhost:port/encrypt' → wysłanie do pliku outputu komendy
curl -s -O -X GET 'http://localhost:port/encrypt'
```

```
curl -X POST 'http://localhost:port/submit' -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"session_id":"id_sesji", "encrypted_data":' "$cat image.enc.b64)" ' "}'
```

Do sprawdzenia listy algorytmów szyfrujących

openssl list -cipher-commands

lub

openssl list -cipher-algorithms

Rodzaje algorytmów i co im trzeba do szczęścia

ecb → klucz szyfrujący

cbc → klucz szyfrujący, wektor inicjalizujący (IV)

OPENSLL SZYFROWANIE używając algorytmu AES-256 w trybie ECB wykorzystując odebrany od serwera klucz szyfrujący

```
openssl enc -AES-256-ecb -in plik_ze_słowem.txt -K klucz_szesnastwowy_szyfrujący -out zaszyfrowany_plik_wyjściowy.enc -base64
```

-base64 / -a -> jeśli wymaga tego zadanie

zamiast pliku ze słowem można wpisać -> <(echo -n słowo)

zamiast wypisania klucza można odczytać z pliku -> \$(cat key)

OPENSLL SZYFROWANIE + iv + brak soli

```
openssl enc -AES-256-cbc -in plik_ze_słowem.txt -out zaszyfrowany_plik_wyjściowy.enc -K klucz_szesnastwowy_szyfrujący -iv wektor_inicjalizujący -nosalt -base64
```

OPENSLL DESZYFROWANIE używając algorytmu AES-256 w trybie ECB

```
openssl enc -d -aes-256-ecb -in zaszyfrowany_plik_base64 -out odszyfrowany_plik_wyjściowy.dec -K klucz_szesnastwowy_szyfrujący -base64
```

musi być na końcu -base64 bo zaszyfrowany plik jest w tym formacie

OPENSLL DESZYFROWANIE pbkdf + pass + iv + brak soli

```
openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:hasło -in zaszyfrowany_plik_base64 -out odszyfrowany_plik_wyjściowy.dec -base64 -iv wektor_inicjalizujący -nosalt
```

`pod -in wchodzi -> <(echo zawartość_encrypted_b64)

OPENSLL DESZYFROWANIE pass + pbkdf2 + iter + nosalt

```
openssl enc -d -aes-256-ecb -in zaszyfrowany_plik_base64.bin -out odszyfrowany_plik_wyjściowy.dec -pass pass:hasło -pbkdf2 -iter liczba_iteracji -nosalt
```

pod -in wchodzi -> echo -n "zaszyfrowany_ciąg_base64" | base64 -d > encrypted.bin

generowanie własnego klucza / iv

openssl rand -hex liczba > key/iv

liczba zależy od ilości bitów z polecenia, gdy klucz musi mieć 192 bity to -> 192/8 = 24, czyli -> openssl rand -hex 24 > key

OPENSLL SZYFROWANIE zdjęć

echo -n "ciąg_base64" | base64 -d > image.png

openssl enc -aria-128-ctr -K key -iv IV -in image.png -out image.enc

base64 image.enc > image.enc.b64 -> w tym pliku trzeba usunąć ręcznie entery bo nie przejdzie w serwerze

lub > base64 -w 0 image.enc > image.enc.b64

OPENSLL DESZYFROWANIE zdjęć

echo -n "ciąg_base64" | base64 -d > image.enc

openssl enc -d -aria-128-cfb -K key -iv IV -in image.enc -out image.png

base64 -w 0 image.png > image.b64

Szyfrowanie asymetryczne

Przydatne curl'e

```
curl -X POST 'http://localhost:port/checkkeys' -H 'accept: application/json' -H 'Content-Type: multipart/form-data' -F 'session_id=id_sesji' -F 'pub_key_file=@pubkey.pem' -F 'priv_key_file=@privkey.pem'
```

```
curl -s -D headers.txt -o privkey.pem -X GET 'http://localhost:port/getprivkey' -H 'accept: application/json' → wysłanie nagłówekó i outputu do pliku
```

```
curl -s -D headers.txt -o response.zip -X GET 'http://127.0.1:3007/decrypt' -H 'accept: application/json'
```

Generowanie pary kluczy RSA (publiczny i prywatny) + eksport do plików + długość klucza 1024 bity

klucz prywatny → openssl genpkey -algorithm RSA -out privkey.pem -pkeyopt rsa_keygen_bits:1024

klucz publiczny → openssl pkey -in privkey.pem -pubout -out pubkey.pem

Generowanie pary kluczy EC - krzywych eliptycznych (publiczny i prywatny) + krzywa eliptyczna prime256v1 + eksport do plików

klucz prywatny → openssl genpkey -algorithm EC -out ecpriv.pem -pkeyopt ec_paramgen_curve:prime256v1

klucz publiczny → openssl ec -in ecpriv.pem -pubout -out ecpub.pem

Wyświetlenie ilu bitowy jest klucz

publiczny EC -> openssl ec -pubin -in ec_public_key.pem -text -noout
prywatny EC -> openssl ec -in ec_private_key.pem -text -noout

uniwersalne:	informacje różne:
openssl pkey -in key.pem -text -noout	openssl x509 -in cert.pem -text -noout
openssl pkey -pubin -in key.pub -text -noout	

Zaszyfrowanie słowa algorytmem RSA-2048 z kluczem publicznym i wybranym trybem paddingu: OAEP

```
openssl pkeyutl -encrypt -pubin -inkey key.pub -pkeyopt rsa_padding_mode:oaep -in plik_ze_słowem -out encrypted.bin
```

pod słowo można też dać -> <(echo -n słowo)

Odkodowanie i odszyfrowanie słowa przy użyciu klucza prywatnego i algorytmu RSA-4096 z paddingiem OAEP

```
openssl pkeyutl -decrypt -inkey private_key.pem -in <(base64 -d encrypted.txt) -pkeyopt rsa_padding_mode:oaep -out decrypted.txt
```

lub

```
base64 -d encrypted.txt > encrypted.bin
openssl pkeyutl -decrypt -inkey private_key.pem -in encrypted.bin -pkeyopt rsa_padding_mode:oaep -out decrypted.txt
```

Podpisanie słowa kluczem prywatnym + przy podpisie parametru paddingu PSS + format base64 + używana funkcja skrótu SHA-256 + długość soli ma być równa z długością skrótu (32B dla SHA-256)

```
openssl dgst -sha256 -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -sign privkey.pem -out słowo.sig word.txt
utworzenie skrótu → openssl dgst -sha256 -binary -out word.sha256 word.txt
zeby sprawdzić ile wynosi długość pliku .sha256 -> ls -l word.sha256
```

```
podpisanie słowa → openssl pkeyutl -sign -in word.sha256 -inkey private_key.pem -pkeyopt digest:sha256 -pkeyopt rsa_padding_mode:pss -pkeyopt rsa_pss_saltlen:32 -out signature.bin
```

```
base64 signature.bin > signature.b64
openssl dgst -sha256 -sign privkey.pem -hex message.txt
```

Weryfikacja podpisu algorytmem RSA-2048 i trybem paddingu PSS + funkcja skrótu SHA-256 + długość soli dopasowana do skrótu

base64 -d signature.b64 > signature.bin

```
openssl dgst -sha256 -verify public_key.pem -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -signature signature.sig word.txt
```

```
przechwycenie wyniku do zmiennej result -> result=$(openssl dgst -sha256 -verify public_key.pem -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -signature signature.bin word.txt | grep -q 'Verified OK' && echo true || echo false)
sprawdzenie czy działa -> echo $result
użycie w curl -> -F "user_verified=$result"
```

Łamanie haseł

Przydatne curl'e

```
curl -X POST 'http://localhost:port/submit' -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"word":"odkodowane_słowo"}
```

Przydatne komendy

man crunch

head -n 5 wordlist.txt

Usuwanie wszystkich kontenerów na raz (usuwa działające i zatrzymane)

podman rm -f \$(podman ps -aq)

Rodzaje hashy z labów po -m

- MD5 -> 0
- SHA-1 -> 100
- SHA-256 -> 1400
- SHA-521 -> 1700
- bcrypt -> 3200

hashcat - rodzaje ataków

hashcat -m rodzaj_hasha -a 0 hash.txt wordlist.txt → podstawowy atak słownikowy

hashcat -m rodzaj_hasha -a 3 hash.txt ????? → brute-force + maska

hashcat -m rodzaj_hasha -a 6 hash.txt wordlist.txt ?d?d -> słownik + słownik
hashcat -m rodzaj_hasha -a 7 hash.txt -1 '!@#%\$&*?'!?!d?d wordlist.txt -> maska + słownik

hashcat sprawdzenie wyniku

hashcat -m rodzaj_hasha hash.txt --show

crunch - generowanie słownika

crunch 3 3 0123456789 -o słownik.txt 3 znaki, każdy jest cyfrą od 0 do 9 (-a 0)

crunch 4 4 ABCDEFGHIJKLMNOPQRSTUVWXYZ -o słownik.txt 4 znaki, każdy jest wielką literą (-a 0)

crunch 8 8 -t pass%%-o wordlist.txt słowo pass z sufiksem: 4 cyfry 0-9, czyli np. pass0001 (-a 3)

crunch 13 -o wordlist.txt -p admin password 123 permutacje słów admin, password i 123, czyli np. 123adminpassword, admin123password (-a 0)

crunch 5 5 -t, @%* -o wordlist.txt 5 znaków -> wielka litera, mała litera, 0-9, 4 i 5 znak to znaki specjalne np. !, @, #, \$ itp.

hashcat - bruteforce

hashcat -m rodzaj_hasha -a 3 hash.txt ?1?!?! 4 znaki -> a-z

hashcat -m rodzaj_hasha -a 3 hash.txt ?u?d?d 4 znaki -> pierwszy A-Z, reszta 0-9

hashcat -m rodzaj_hasha -a 3 hash.txt -1 abc -2 468 -3 %* '?12?3?' 3 znaki -> pierwszy (a,b,c), drugi (4,6,8), trzeci (*,%,?)

hashcat -m rodzaj_hasha -a 3 hash.txt '00:14:22:ff:?:?:?' MAC zaczynający się 00:14:22:ff:?:?:

hashcat -m rodzaj_hasha -a 6 hash.txt wordlist.txt ?d?d to słowo ze słownika, do którego dodano dwie cyfry(0-9) na jego końcu

hashcat -m rodzaj_hasha -a 7 hash.txt -1 '!@#%\$&*?'!?!d?d wordlist.txt to słowo ze słownika, do którego dodano na początku literę(a-z), cyfrę(0-9) i znak specjalny(!, @, #, \$, %, &,*)

plik rule.txt do tworzenia własnych zasad

1. hasło ze słowami z pliku przekształcone jako (a->@, e->3, i->1)

sa@se3s!l
tUmaccnie:

- s - do zamiiany
- sa@ - zamiień 'a' na '@'
- se3 - zamiień 'e' na '3'
- si! - zamiień 'i' na '!'

echo -n "sa@se3s!l" > rule.txt

ZŁAMANIE HASŁA V

hashcat -m rodzaj_hasha -a 0 hash.txt wordlist.txt -r rule.txt

2. hasło ze słowami z pliku przekształcone jako (pierwsza litera wielka, zamienione litery a->4 e->3 o->0, na końcu słowa dodany znak _ ,bieżący rok i znak !)

echo -n "T0 sa4 se3 s0 \$ _ \$2 \$0 \$2 \$5 \$!" > rule.txt

3. hasło ze słowami z dodanym na początku ! i # na końcu

echo -n "!! \$#" > rule.txt

Najważniejsze skróty do pliku z zasadami (rule.txt)

Pierwsza duża:	T0
Pierwsza mała:	t0
Ostatnia duża:	T\$
Ostatnia mała:	t\$
Wszystkie duże:	T\$
Wszystkie małe:	Tl
Zamiień litery na przemian:	Ta
Zmiana a=0:	sa0
Zmiana pierwszego a na 0:	rza0
Zmiana ostatniego a na 0:	lza0
Dodaj X na końcu:	\$X
Dodaj X na początek:	*X
Dodaje ciąg znaków:	"pass123
Usuń pierwszy znak:	D0
Usuń drugi znak:	D1
Usuń ostatni znak:	D\$
Skród do długości N:	JN (np. [js]
Odwróć:	r (np. kot -> tok)
Powróź słowa:	pk (np. pz: kot -> kotok)
Duplikuj pierwszą literę:	d (np. kot -> kkot)
Wstaw znak X na pozycji n:	inX (np. i!@ -> !k@t, i\$! -> k\$!t)
Przykłady:	
Podmiana liter na liczby + pierwsza litera DUŻA -> T0 sa0 se3 s!l s00	
Dodanie roku na końcu + duża pierwsza litera -> T0 \$2 \$0 \$2 \$3. # dodaje "2023"	
Imię + ! + liczba na końcu -> T0 \$! \$1	
Pierwsza DUŻA + zamiany leet + wykrzyknik -> T0 sa0 se3 s!l \$!	

GPG GPG

Przydatne curre

curl -X POST 'http://localhost:port/submit/public' -F "file=@pub.key"

curl -X POST 'http://localhost:port/submit/public' -F "file=tekst_z_pliku"

curl -X POST 'http://localhost:port/submit' -H 'Content-Type: application/json' -d '{"session_id":"id_sesji", "decrypted_text":""}' "\$(cat decrypted.txt)"'

Przydatne komendy

gpg --list-keys wyświetlwnie wszystkich kluczy

gpg --list-public-keys wyświetlwnie publicznych kluczy

gpg --list-secret-keys wyświetlwnie prywatnych kluczy

gpg --list-keys --fingerprint

gpg --list-keys --keyid-format long

gpg --delete-secret-keys id_klucza usuwanie klucza prywatnego

gpg --delete-keys id_klucza usuwanie klucza publicznego

gpg --delete-secret-and-public-key id_klucza usuwanie obu kluczy za jednym razem

GENEROWANIE kluczy GPG

gpg --full-generate-key

EKSPORT klucza publicznego do pliku

gpg --armor --export id_klucza > pub.key

EKSPORT klucza prywatnego do pliku

gpg --armor --export-secret-key id_klucza > priv.key

IMPORT klucza publicznego / prywatnego do słownika zbioru

gpg --import key.asc

ODSZYFROWANIE pliku używając algorytmu CAMELLIA128 i hasła z pliku

gpg --cipher-algo CAMELLIA128 --decrypt encrypted.txt > decrypted.txt

wpisać hasło z passphrase.txt

ZYFROWANIE pliku ze słowem za pomocą algorytmu AES128 i pliku z hasłem + base64(armor)

gpg --symmetric --cipher-algo AES128 --armor --output encrypted.txt word_to_encrypt.txt

ZYFROWANIE pliku ze słowem kluczem publicznym + base64 - bez hasła

gpg --encrypt --recipient id_klucza --armor --output encrypted.txt word.txt

ODSZYFROWANIE pliku ze słowem za pomocą pobranego klucza prywatnego

gpg --decrypt --recipient id_klucza --output decrypted_word.txt encrypted_word.txt

PODPIS pliku ze słowem za pomocą pobranego klucza publicznego

chodzi o import prywatnego klucza bo tylko prywatnym się podpisuje

gpg -u id_klucza --sign --output word.txt.gpg word.txt

gpg -u id_klucza --detach-sign --output word.txt.sig word.txt utworzenie oddzielnego podpisu

gpg -u id_klucza --clearsign --output word.txt.sig word.txt utworzenie czytelnego podpisu

PODPIS importowanego klucza publicznego swoim prywatnym + eksport

- gpg --full-generate-key
- gpg --armor --export id_klucza > pub.key
- gpg --import server_key.asc
- gpg --local-user id_klucza podpisującego --sign-key id_klucza_do_podpisania podpis
- gpg --export --armor id_klucza_do_podpisania > signed_key.asc

WERYFIKACJA PODPISU za pomocą importowanego klucza publicznego

gpg --verify word.txt.gpg

Powtorzenie

ZAD1

Zahaszuj słowo Linux za pomocą algorytmu MD5.

echo -n "Linux" | openssl dgst -md5 > word.txt

echo -n "Linux" | md5sum > word.txt

ZAD2

Zahaszuj słowo Linux za pomocą algorytmu SHA-256.

echo -n "Linux" | openssl dgst -SHA-256 > word.txt

echo -n "Linux" | sha256sum > word.txt

ZAD3

Zakoduj słowo Linux za pomocą kodowania base64.

echo -n "Linux" | base64 > word.b64

- echo -n "Linux" > word.txt
- base64 word.txt > word.b64

ZAD4

Odkoduj słowo VU1DUw== zakodowane za pomocą base64.

- echo -n "VU1DUw==" > encoded_word.b64
- cat encoded_word.b64 | base64 -d > decoded_word.txt

echo -n "VU1DUw==" | base64 -d > decoded_word.txt

ZAD5

Zaszzyfruj słowo Linux za pomocą algorytmu AES-256-ECB i klucza

e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e.

openssl enc -AES-256-ecb -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e -in <(echo -n "Linux") -out word.enc

- echo -n "Linux" > word.txt
- openssl enc -AES-256-ecb -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e -in word.txt -out word.enc

ZAD6

Zaszzyfruj słowo Linux za pomocą AES-256-CFB, klucza

e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e oraz IV 83c468dc477543a6906912b6a1344416.

openssl enc -AES-256-CFB -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e -iv 83c468dc477543a6906912b6a1344416 -in <(echo -n "Linux") -out word.enc

ZAD7

Zaszzyfruj słowo Linux za pomocą AES-256-CFB, klucza

e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e oraz IV 83c468dc477543a6906912b6a1344416. Wynik szyfrowania zakoduj za pomocą kodowania base64.

openssl enc -AES-256-CFB -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e -iv 83c468dc477543a6906912b6a1344416 -in <(echo -n "Linux") -out word.enc -base64

openssl enc -AES-256-CFB -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e -iv 83c468dc477543a6906912b6a1344416 -in <(echo -n "Linux") -out word.enc -a

ZAD8

Słowo f6NB8w== zostało zaszyfrowane algorytmem AES-256-CFB, kluczem e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e oraz IV 83c468dc477543a6906912b6a1344416, a następnie zakodowane za pomocą kodowania base64. Odszyfruj i odkoduj słowo.

- echo -n "f6NB8w==" | base64 -d > word.enc
- openssl enc -d -AES-256-CFB -in word.enc -out decrypted_word.txt -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e -iv 83c468dc477543a6906912b6a1344416

ZAD9

Wygeneruj klucz szyfrujący o długości 128 bitów.

openssl rand -hex 16 > key

ZAD10

Zaszzyfruj słowo Linux za pomocą AES-128-CBC, bez soli, za pomocą generowania klucza to PBKDF2, z liczbą iteracji równą 256.

openssl enc -AES-128-CBC -in <(echo -n "Linux") -out word.enc -nosalt -pass pass:Ubuntu -pbkdf2 -iter 256 -base64

ZAD11

Wygeneruj parę kluczy (publiczny i prywatny), klucze mają 2048 bitów. Wyeksportuj klucze do pliku.

openssl genkey -algorithm RSA -out privkey.pem -pkeyopt rsa_keygen_bits:2048
openssl pkey -in privkey.pem -pubout -out pubkey.pem

ZAD12

Wygeneruj parę kluczy (publiczny i prywatny), klucze mają 2048 bitów. Wyeksportuj klucze do pliku. Zaszzyfruj słowo Linux kluczem publicznym i odszyfruj prywatnym. Użyj paddingu OAEP.

- openssl genkey -algorithm RSA -out priv.pem -pkeyopt rsa_keygen_bits:2048
- openssl pkey -in priv.pem -pubout -out pub.pem
- openssl pkeyutl -encrypt -pubin -inkey pub.pem -out word.enc
rsa_padding_mode:oaep -in <(echo -n "Linux") -out word.enc
- openssl pkeyutl -decrypt -inkey priv.pem -pkeyopt rsa_padding_mode:oaep -in word.enc -out word.txt

ZAD13

Wygeneruj parę kluczy (publiczny i prywatny), klucze mają 2048 bitów. Wyeksportuj klucze do pliku. Zapisz do pliku słowo Linux. Podpisz plik używając klucza prywatnego, z paddingiem PSS: hash SHA-256, długość soli powinna odpowiadać długości funkcji skrótu. Zweryfikuj wygenerowany podpis.

- openssl genkey -algorithm RSA -out privkey.pem -pkeyopt rsa_keygen_bits:2048
- openssl pkey -in privkey.pem -pubout -out pubkey.pem
- echo -n "Linux" > word.txt
- openssl dgst -sha256 -binary -out word.sha256 word.txt
- openssl pkeyutl -sign -in word.sha256 -inkey privkey.pem -pkeyopt digest:sha256 -pkeyopt rsa_padding_mode:pss -pkeyopt rsa_pss_saltlen:32 -out signature.bin
- openssl dgst -sha256 -verify pubkey.pem -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -signature signature.bin word.txt

ZAD14

Złam hash MD5: e2fc714c4727ee9395f324cd2e7f331f za pomocą:

- ataku słownikowego,
- generując swój własny słownik, widząc, że hasło ma 4 znaki i składa się z małych liter a-z

echo -n "e2fc714c4727ee9395f324cd2e7f331f" > hash.txt

crunch 4 4 -t @@@@ -o słownik.txt

hashcat -m 0 -a 0 hash.txt słownik.txt

hashcat -m 0 -a 3 hash.txt ?1?!?!
hashcat -m 0 hash.txt --show

ZAD15

Wygeneruj parę kluczy GPG dla maila student@umcs.pl. Klucz RSA, 1024 bity. Wyeksportuj klucz publiczny i prywatny do pliku.

- gpg --full-generate-key (1 -> 1024 -> student@umcs.pl)
- gpg --armor --export id_klucza > pub.key
- gpg --armor --export-secret-key id_klucza > priv.key

ZAD16

Zaszzyfruj słowo Linux algorytmem AES z kluczem 128 bity, z hasłem Ubuntu używając GPG. Wynik zapisz do pliku. Odszyfruj plik.

- gpg --symmetric --cipher-algo AES128 --armor --output word.enc <(echo -n "Linux")
- wpisać hasło Ubuntu w okienku
- gpg --decrypt --cipher-algo AES128 --armor --output wprd.txt word.enc

ZAD17

Wygeneruj parę kluczy RSA (GPG) z domyślnymi parametrami. Zaszzyfruj słowo Linux przy użyciu klucza publicznego. Wynik szyfrowania zapisz do pliku. Odszyfruj plik.

- gpg --full-generate-key
- gpg --armor --export id_klucza > pub.key
- gpg --armor --export-secret-key id_klucza > priv.key
- gpg --encrypt --recipient id_klucza --armor --output word.enc <(echo -n "Linux")
- gpg --decrypt --recipient id_klucza --output word.dec word.enc

ZAD18

Wygeneruj parę kluczy RSA (GPG) z domyślnymi parametrami. Zapisz słowo Linux do pliku, podpisz plik przy użyciu klucza publicznego. Użyj kodowania base64. Wynik podpisywania zapisz do pliku. Zweryfikuj podpisany plik.

- gpg --full-generate-key
- gpg --armor --export id_klucza > pub.key
- gpg --armor --export-secret-key id_klucza > priv.key
- echo -n "Linux" > word.txt
- gpg -u id_klucza --armor --sign --output word.sig word.txt
- gpg --verify word.sig